

Joel Sivanish

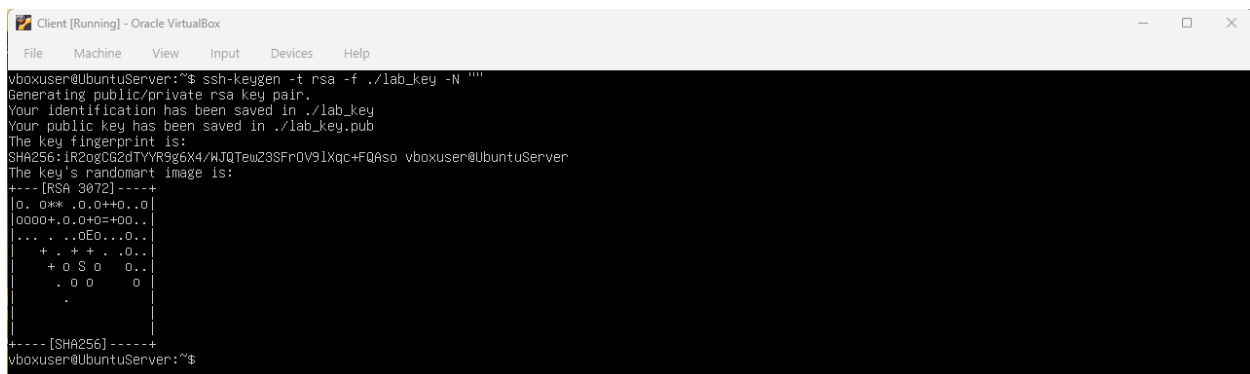
Professor Kaur

Lab 4 - Secure Data Transmission Using AES Encryption

2/10/2026

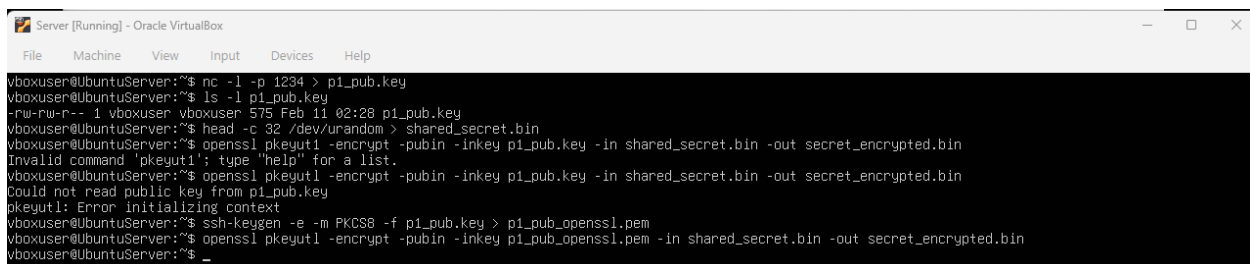
Code and Technical Implementation:

- **RSA Key Generation (Client):** Used to create the asymmetric key pair for the initial secure handshake.
 - `ssh-keygen -t rsa -f ./lab_key -N ""`



```
Client [Running] - Oracle VirtualBox
File Machine View Input Devices Help
vboxuser@UbuntuServer:~$ ssh-keygen -t rsa -f ./lab_key -N ""
Generating public/private rsa key pair.
Your identification has been saved in ./lab_key
Your public key has been saved in ./lab_key.pub
The key fingerprint is:
SHA256:IR2ogCG2dTVR9gGx4/HJQTewZ3SFr0V9lXqc+FQAsO vboxuser@UbuntuServer
The key's randomart image is:
+--[RSA 3072]-----+
|o..o**..o.o+o+o..o|
|o000+o.o+o+o+o+o..|
|... ..oEo...o..|
|+..+..+..+..+..+..|
|+oSo..o..|
|.o.o..o..|
|..|
+-----[SHA256]-----+
vboxuser@UbuntuServer:~$
```

- **Public Key Conversion (Server):** Converted the SSH public key to PKCS#8 format to allow OpenSSL to process the encryption.
 - `ssh-keygen -e -m PKCS8 -f p1_pub.key > p1_pub_openssl.pem`



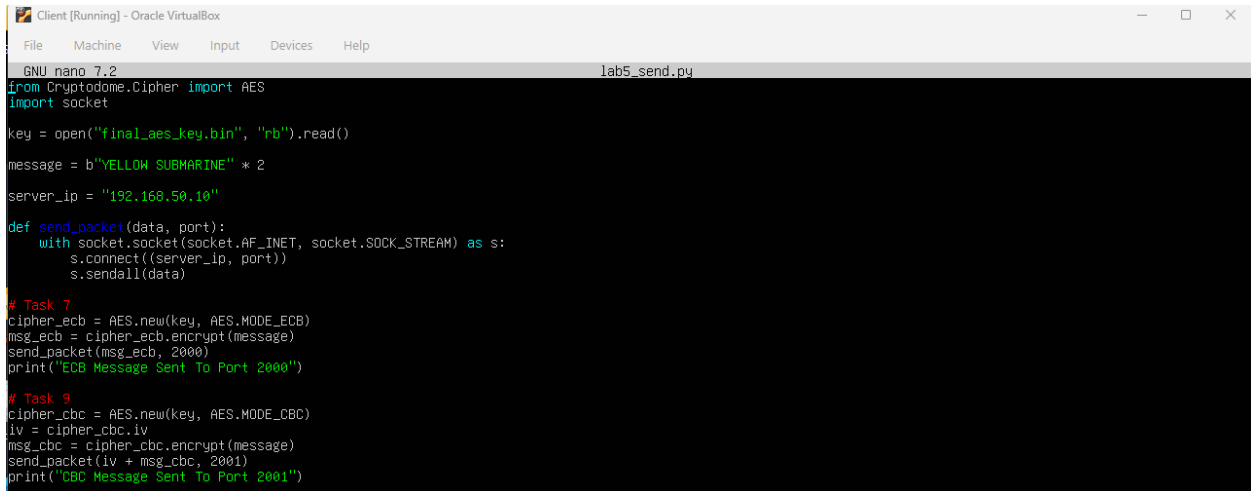
```
Server [Running] - Oracle VirtualBox
File Machine View Input Devices Help
vboxuser@UbuntuServer:~$ nc -l -p 1234 > p1_pub.key
vboxuser@UbuntuServer:~$ ls -l p1_pub.key
-rw-rw-r-- 1 vboxuser vboxuser 575 Feb 11 02:28 p1_pub.key
vboxuser@UbuntuServer:~$ head -c 32 /dev/urandom > shared_secret.bin
vboxuser@UbuntuServer:~$ openssl pkeyutl -encrypt -pubin -inkey p1_pub.key -in shared_secret.bin -out secret_encrypted.bin
Invalid command "pkeyutl"; type "help" for a list.
vboxuser@UbuntuServer:~$ openssl pkeyutl -encrypt -pubin -inkey p1_pub.key -in shared_secret.bin -out secret_encrypted.bin
Could not read public key from p1_pub.key
pkeyutl: Error initializing context
vboxuser@UbuntuServer:~$ ssh-keygen -e -m PKCS8 -f p1_pub.key > p1_pub_openssl.pem
vboxuser@UbuntuServer:~$ openssl pkeyutl -encrypt -pubin -inkey p1_pub_openssl.pem -in shared_secret.bin -out secret_encrypted.bin
vboxuser@UbuntuServer:~$
```

- **AES Key Recovery (Client):** Used the private RSA key to decrypt the shared secret received from the server, resulting in a 32-byte (256-bit) AES key.
 - `openssl pkeyutl -decrypt -inkey lab_key -in secret_received.bin -out final_aes_key.bin`



```
Client [Running] - Oracle VirtualBox
File Machine View Input Devices Help
vboxuser@UbuntuServer:~$ ls -l final_aes_key.bin
-rw-rw-r-- 1 vboxuser vboxuser 32 Feb 11 02:55 final_aes_key.bin
vboxuser@UbuntuServer:~$
```

- Pattern Leak Transmission Script (Python): This script utilized the PyCryptodome library to send identical plaintext blocks ("YELLOW SUBMARINE") via both ECB and CBC modes to visualize security differences.



```
Client [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 7.2 lab5_send.py
from Cryptodome.Cipher import AES
import socket

key = open("final_aes_key.bin", "rb").read()

message = b"YELLOW SUBMARINE" * 2

server_ip = "192.168.50.10"

def send_packet(data, port):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((server_ip, port))
        s.sendall(data)

# Task 7
cipher_ecb = AES.new(key, AES.MODE_ECB)
msg_ecb = cipher_ecb.encrypt(message)
send_packet(msg_ecb, 2000)
print("ECB Message Sent To Port 2000")

# Task 9
cipher_cbc = AES.new(key, AES.MODE_CBC)
iv = cipher_cbc.iv
msg_cbc = cipher_cbc.encrypt(message)
send_packet(iv + msg_cbc, 2001)
print("CBC Message Sent To Port 2001")
```

1. During the public key and shared secret exchange, what specific values are visible to an observer? Explain why an attacker cannot derive the final AES key despite seeing these transmissions.

- An observer can see the plaintext RSA Public Key and the RSA-encrypted shared secret. However, an attacker cannot derive the final AES key because the shared secret is protected by asymmetric encryption. Only the holder of the corresponding Private Key (the Client) can decrypt the secret to reveal the actual AES key used for the session.

Wireshark · Packet 23 · lab5_p1.pcap

Protocol: IPv4 (0x0800)
 Reserved: 0000
 Interface index: 3
 Link-layer address type: Ethernet (1)
 Packet type: Sent by us (4)
 Link-layer address length: 6
 Source: PCSSystemtec_86:6f:0d (08:00:27:86:6f:0d)
 Unused: 0000

> Internet Protocol Version 4, Src: 192.168.50.11, Dst: 192.168.50.10
 > Transmission Control Protocol, Src Port: 34726, Dst Port: 1234, Seq: 1, Ack: 1, Len: 575
 ✓ Data (575 bytes)

Data [...]: 7373682d727361204141414142334e7a61433179633245414141414441514142414141426751444b6...

0000	08 00 00 00 00 00 00 03	00 01 04 06 08 00 27 86	
0010	6f 0d 00 00 45 00 02 73	62 48 40 00 40 06 f0 d6	o...E...s bH@...@...
0020	c0 a8 32 0b c0 a8 32 0a	87 a6 04 d2 44 ca 70 9e	--2...2-D.p.
0030	a0 42 4c a6 80 18 01 f6	e7 cb 00 00 01 01 08 0a	-BL.....
0040	91 64 59 54 a0 97 bd d1	73 73 68 2d 72 73 61 20	-dYT.... ssh-rsa
0050	41 41 41 41 42 33 4e 7a	61 43 31 79 63 32 45 41	AAAAB3Nz aC1yc2EA
0060	41 41 41 44 41 51 41 42	41 41 41 42 67 51 44 4b	AAADAQAB AAABgQDK
0070	6a 76 52 67 42 35 79 4a	39 42 39 45 42 4c 38 6c	jvRgB5yJ 9B9EBL8l
0080	36 4d 50 51 77 79 4a 36	74 46 41 47 2f 79 37 73	6MPQwyJ6 tFAG/y7s
0090	6c 35 68 66 7a 6a 4a 31	57 7a 57 44 7a 51 69 51	l5hfzjJ1 WzWDzQiQ
00a0	37 69 35 76 2b 72 5a 46	59 6b 64 4a 4f 4e 72 59	7i5v+rZF YkdJONrY
00b0	47 70 6d 79 6e 44 70 67	6f 4b 41 39 54 56 69 6e	GpmynDpg oKA9TVin
00c0	35 66 37 53 34 4d 52 78	4d 41 4e 34 2f 52 69 6d	5f7S4MRx MAN4/Rim
00d0	35 65 2f 4b 52 61 2b 71	49 38 6a 66 62 5a 67 31	5e/KRa+q I8jfbzg1
00e0	33 6c 57 2f 50 38 65 59	4a 6f 37 65 56 79 32 63	3lw/P8eY Jo7eVy2c
00f0	74 71 5a 54 59 6b 33 6f	2f 31 56 78 4a 55 44 66	tqZTYk3o /1VxJUDf
0100	2b 32 44 45 37 35 47 54	6b 41 72 73 76 43 6f 2b	+2DE75GT kArsvCo+
0110	34 70 39 6f 76 35 4b 71	64 66 76 55 45 47 51 50	4p9ov5Kq dfvUEGQP
0120	58 78 76 50 30 5a 56 63	6e 36 69 58 53 4b 61 71	XxvP0ZVc n6iXSKaq
0130	73 64 68 4b 66 6f 6f 2f	76 4e 52 2f 79 76 69 55	sdhKfoo/ vNR/yviU
0140	33 59 70 45 36 70 38 56	62 77 51 44 67 31 73 59	3YpE6p8V bwQDg1sY
0150	54 44 45 69 79 59 70 72	59 78 36 42 6b 38 45 79	TDEiyYpr Yx6Bk8Ey
0160	6f 77 71 6a 35 72 35 77	53 53 59 78 4a 75 61 44	owqj5r5w SSYxJuaD
0170	75 64 33 50 5a 32 50 48	57 5a 67 42 31 5a 38 42	ud3PZ2PH WZgB1Z8B
0180	43 34 76 74 50 35 4b 62	71 44 38 2b 64 74 4a 50	C4vtP5Kb qD8+dtJP
0190	71 68 7a 56 78 35 79 73	77 66 45 73 37 58 35 68	qhzVx5ys wfEs7X5h
01a0	4a 46 76 64 4d 6b 6b 5a	6a 61 42 50 4d 56 56 31	JFvdMkkZ jaBPMVV1
01b0	50 33 67 65 47 78 4c 49	4f 33 34 48 34 6e 4d 77	P3geGxLI 034H4nMw
01c0	72 75 45 34 6a 34 2b 55	4e 6e 41 4b 70 4c 4f 35	ruE4j4+U NnAKpLO5
01d0	4d 72 69 49 44 5a 77 44	61 53 43 42 71 79 77 59	MriIDZwD aSCBqywY
01e0	52 47 7a 47 64 6c 4a 52	73 36 71 2b 58 6d 75 32	RGzGdlJR s6q+Xmu2
01f0	61 59 76 38 64 69 4b 4b	35 46 52 7a 6d 6b 39 2b	aYv8diKK 5FRzmk9+
0200	61 52 73 6e 53 4e 77 47	61 2f 37 70 62 6e 43 4d	aRsnSNwG a/7pbnCM
0210	4a 4a 62 41 62 42 4c 74	64 48 42 6d 50 45 52 4f	JJbAbBLt dHBmPERO
0220	35 56 42 70 44 63 30 63	61 66 66 43 53 32 77 72	5VBpDc0c affCS2wr
0230	70 52 69 65 77 55 61 75	58 6c 6c 6c 4a 70 30 6b	pRiewUau X1llJp0k
0240	6c 49 55 39 6b 70 73 4a	56 2b 59 32 5a 46 68 45	lIU9kpsJ V+Y2ZFhE
0250	46 2f 2b 71 6c 41 6b 46	7a 47 58 4c 2f 4e 30 75	F/+qlAkF zGXL/N0u
0260	39 68 76 4a 69 36 6a 78	64 69 75 6a 6a 6d 6b 3d	9hvJi6jx diujjmk=
0270	20 76 62 6f 78 75 73 65	72 40 55 62 75 6e 74 75	vboxuse r@Ubuntu
0280	53 65 72 76 65 72 0a		Server:

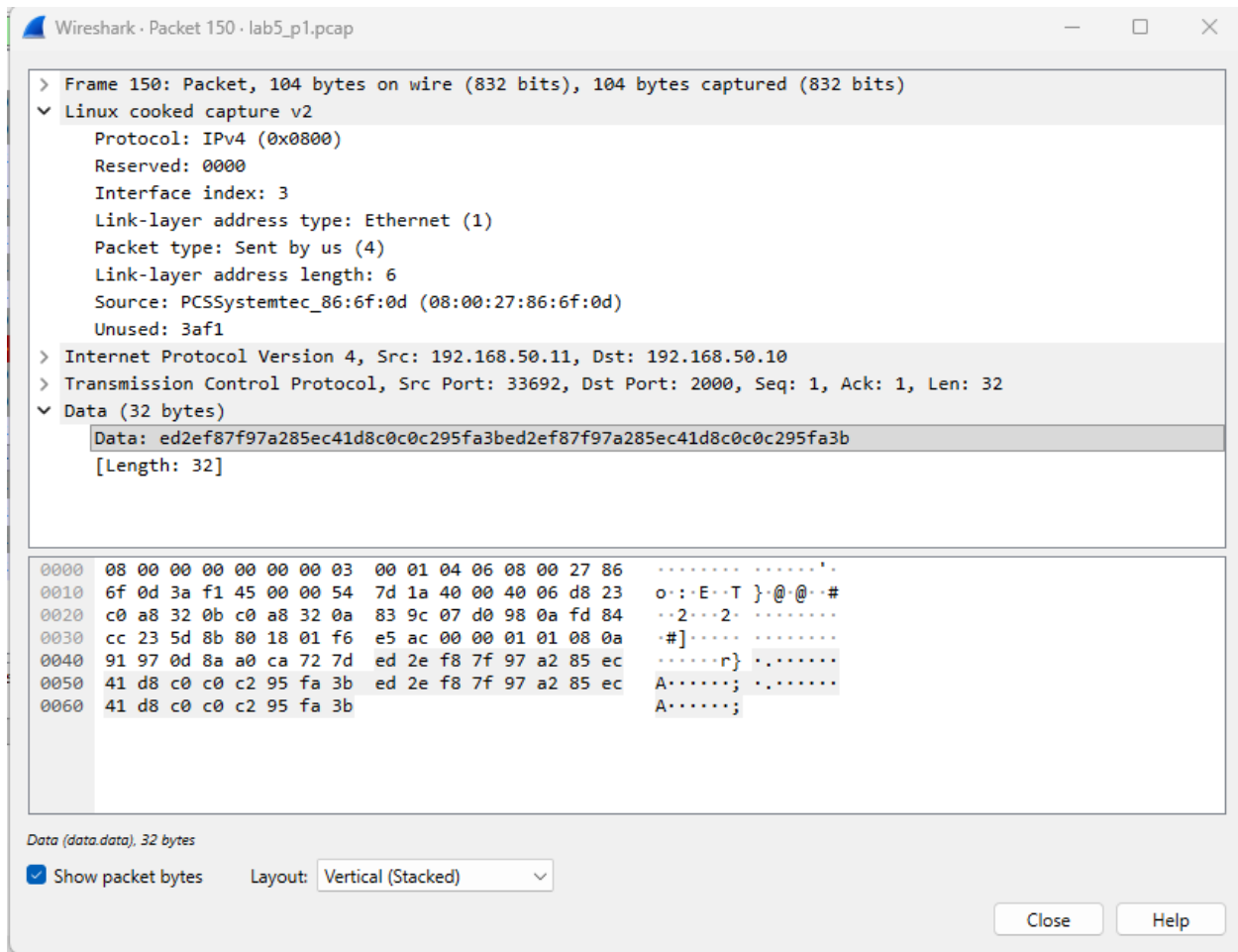
Data (data.data), 575 bytes

☒ Show packet bytes Layout: Vertical (Stacked) ▾

Close Help

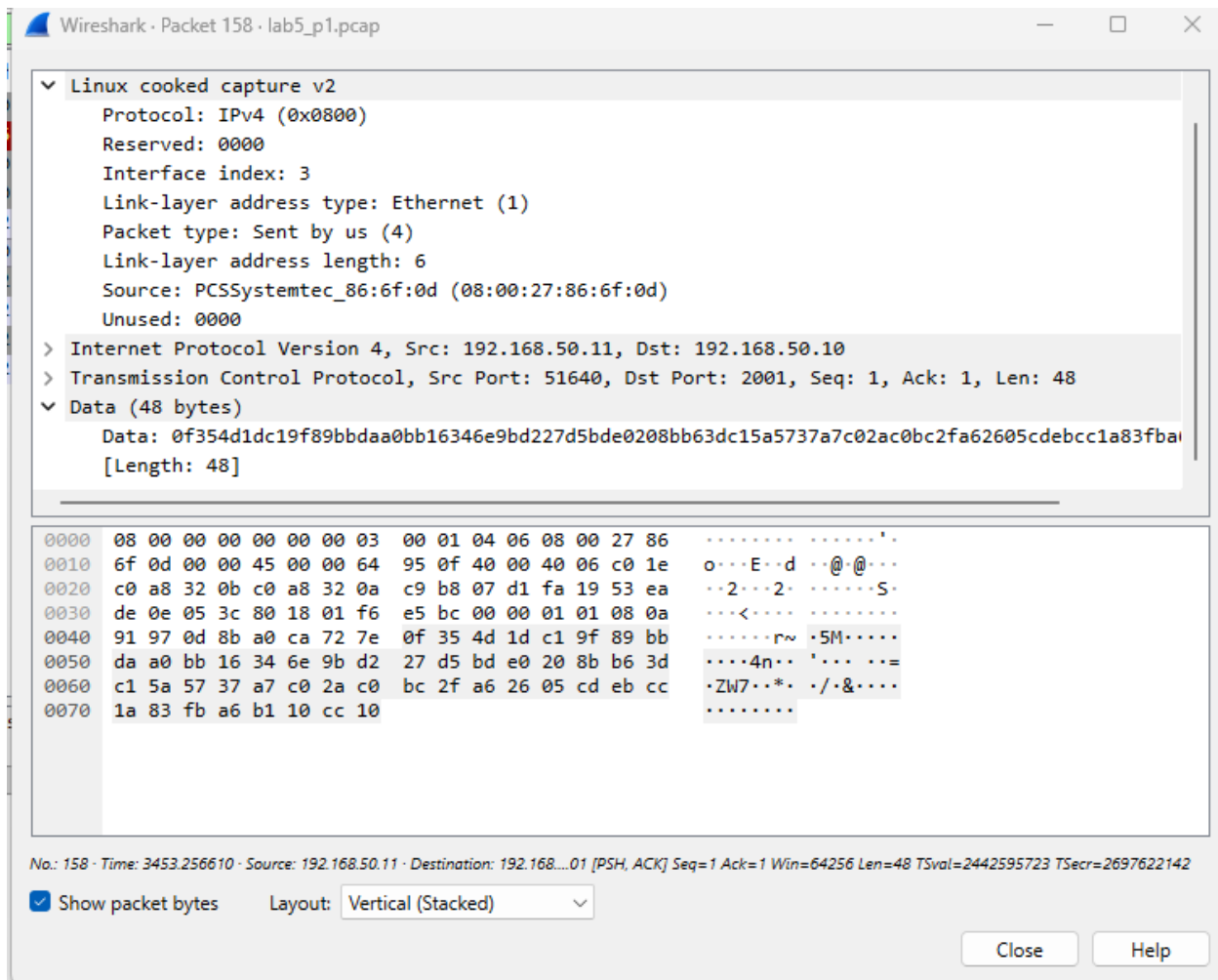
2. Identify a repetitive pattern in your ECB-encrypted traffic. What does this pattern "leak" to an observer about the original plaintext, even without the decryption key?

- In the ECB capture, the hex dump reveals that the first 16 bytes of ciphertext are identical to the second 16 bytes. This "leaks" the fact that the original message contained repeating data blocks, allowing an observer to perform frequency analysis or identify structured data patterns without needing the key.



3. In the CBC exchange, how does the visual output of identical messages differ from the ECB exchange? Explain the role of the Initialization Vector (IV) in preventing an observer from identifying repeated messages.

- Unlike ECB, the CBC output appears completely randomized with no visible patterns. The IV serves as a unique starting block for the first encryption step; because the IV is unique for every session, identical plaintext blocks result in entirely different ciphertext, ensuring that patterns are never revealed.

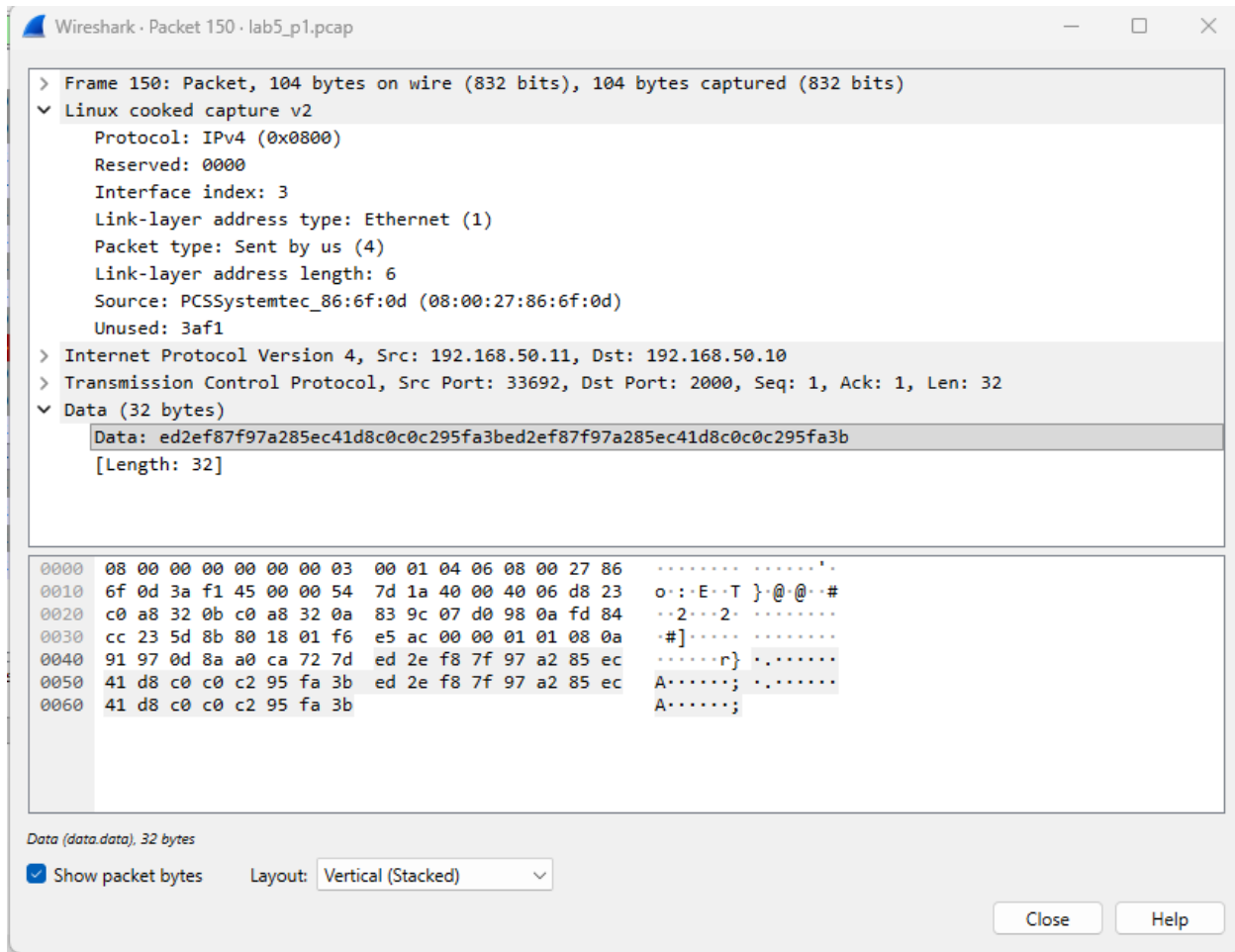


4. Beyond the encrypted payload, what can an attacker infer about the nature of your communication (e.g., protocol, length, or intent) by looking only at the unencrypted packet headers?

- Packet headers reveal the Source/Destination IP addresses, the transport protocol (TCP), and the specific port numbers (2000 and 2001). This allows an attacker to map the network topology and identify that specific cryptographic services or custom applications are active on those ports.

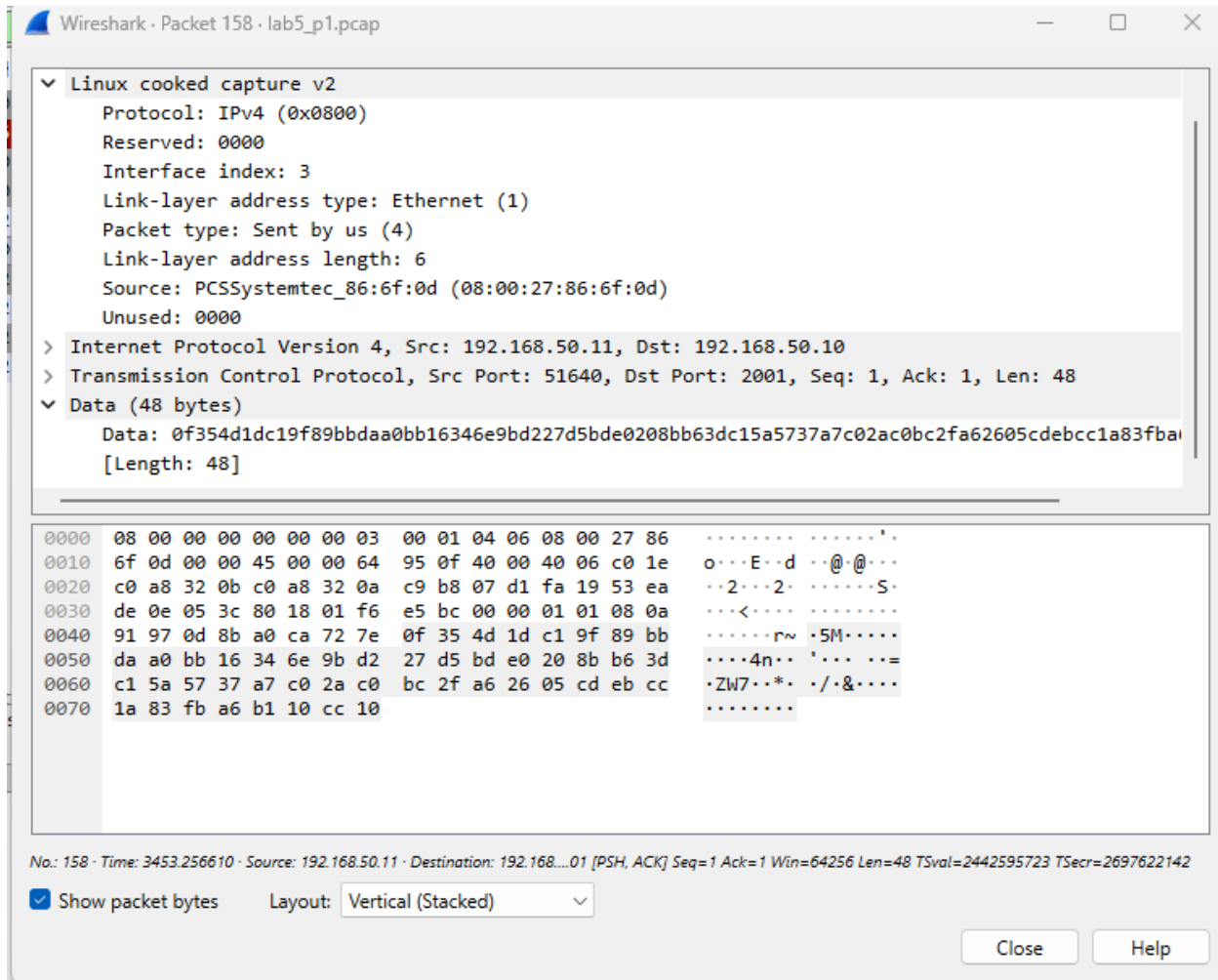
5. Locate a sequence of identical plaintext blocks in your ECB capture. How do the resulting ciphertext blocks appear in the hex dump, and what does this "determinism" reveal to an observer about the underlying structure of your data?

- The blocks appear as identical hex strings: ed2ef87f97a285ec41d8c0c0c295fa3b. This determinism reveals that the underlying data is highly structured or repetitive, which is a major security vulnerability as it can expose the nature of the files or protocols being used (such as headers in a standard file format).



6. In your CBC capture, can the IV be seen? If an attacker can see the IV but not the key, does this compromise the confidentiality of the message? Why or why not?

- The IV is clearly visible as the first 16 bytes of the Data payload. This does not compromise confidentiality because the IV is not a secret; it is simply a "randomizer." Without the 256-bit AES key, the attacker still cannot decrypt the "chain" of blocks following the IV.



7. Does the size of your encrypted packet match the exact size of your original plaintext message? Why might this differ?

- The encrypted packet is often larger (in this lab, 48 bytes for CBC vs 32 bytes for the plaintext). This difference is due to the addition of the 16-byte Initialization Vector (IV) and any necessary padding required to ensure the plaintext fits into the 16-byte block size required by AES.

8. Based on the port numbers and packet sequences seen in your tcpdump, could an observer identify the specific application or tool you used to send these messages? Justify your answer using evidence from your capture.

- Yes, an observer could likely identify the use of netcat (nc) for the handshake based on the characteristic one-way TCP streams. Furthermore, the use of ports 2000 and 2001, combined with the successful execution shown in terminal logs, provides evidence of a custom script or a lab-specific tool designed to isolate different encryption modes.

```
vboxuser@UbuntuServer:~$ python3 lab5_send.py
ECB Message Sent To Port 2000
CBC Message Sent To Port 2001
vboxuser@UbuntuServer:~$
```

```
Server [Running] - Oracle VirtualBox
File Machine View Input Devices Help
vboxuser@UbuntuServer:~$ nc -l -p 1234 > p1_pub.key
vboxuser@UbuntuServer:~$ ls -l p1_pub.key
-rw-rw-r-- 1 vboxuser vboxuser 575 Feb 11 02:28 p1_pub.key
vboxuser@UbuntuServer:~$
```